Smart Home Alarm Control System: Distributed Cluster Design and Implementation Report

Eduard Toni Alexandru

October 28, 2026

Abstract

This report describes the analysis, design and implementation of a smart home alarm control system built with the Apache Pekko framework and deployed on a Pekko Cluster. Sensors, keypads, sirens and the alarm control unit are actors that may run on different cluster nodes and interact only through messages. The control unit runs as a Cluster Singleton, so exactly one instance exists in the whole cluster; every new instance, whether created after a crash, a restart or the loss of its node, starts in a safe recovery mode and requires the correct PIN before returning to the normal state machine. The behaviour is verified by automated tests that run three real cluster nodes connected over the network.

1 Problem Analysis

The smart home alarm domain is inherently concurrent, distributed and reactive. From a concurrent and distributed systems point of view, the main aspects are the following.

- **Autonomous Concurrent Entities:** Sensors, keypads, sirens and the control unit operate independently. Several sensors can detect events at the same time, and a user can type on the keypad while sensor events are in flight.
- **Asynchronous Communication:** The components cannot share memory or rely on blocking calls. Message passing gives loose coupling and avoids race conditions.
- **State-Dependent Message Handling:** The same message has different effects depending on the current state: for example, a sensor event is only logged when the system is disarmed, but starts the entry delay when it is armed.

- **Temporal Constraints:** Exit and entry delays require non-blocking timeouts that trigger state transitions autonomously.
- **Distribution and Location Transparency:** Sensors, keypads and the control unit may run on different nodes. A sensor must be able to reach the control unit without knowing on which node it runs, and every message that crosses nodes must be serialized.
- **A Single Control Entity:** The alarm state must be decided in one place. If two control units were active at the same time, sensors and keypads could talk to different ones, each with its own, inconsistent state.
- **Partial Failures:** A node can crash while the others keep running. If the control unit is lost, a new one must be created, but its previous state (armed or not, active zones, pending timers) is lost with it. The new instance must not guess: assuming *disarmed* would ignore a real intrusion, and assuming *armed* could sound a false alarm.

1.1 Functional Requirements

- **System States:**
  - *Disarmed:* sensors do not trigger alarms; normal activity is allowed.
  - *Exit Delay:* entered after arming with the correct PIN. Sensors are still inactive, so that the user can leave. After a fixed timeout the system becomes Armed.
  - *Armed:* sensors are active; an intrusion in an active zone starts the Entry Delay.
  - *Entry Delay:* a countdown starts. The correct PIN before the timeout leads to Disarmed; otherwise the system goes to Alarm.
  - *Alarm:* the siren is activated; only the correct PIN stops it and leads to Disarmed.
- **User Interaction:** the keypad is used to arm, to disarm and to stop the alarm. The PIN is validated before any state transition.
- **Sensors:** each sensor has a `sensorId`; motion sensors and door/window sensors are supported. Sensor events are sent to the control unit. In Disarmed and Exit Delay they are only logged, in Armed they start the Entry Delay, in Entry Delay and Alarm they have no additional effect.
- **Timing Constraints:** configurable exit delay (e.g. 20–60 s) and entry delay (e.g. 10–30 s); timeouts trigger the transitions automatically.

- **Zones:** every sensor belongs to a zone. The user can arm all zones (full arming) or only some of them (partial arming, e.g. night mode with the upper floor inactive); sensors in inactive zones are ignored.
- **Cluster Architecture:** the system runs on at least three cluster nodes; sensors, keypads and the control unit may run on different nodes; communication happens only through actor messages.
- **Failure and Recovery:** when the control unit is restarted or recreated it enters a safe recovery mode. It must assume neither Disarmed nor Armed, sensor events are only logged, and a correct PIN is required to return to Disarmed.

2 Design Strategy

The system adopts the Actor Model through the classic Apache Pekko API. The alarm logic is the same as in the single-node version; the cluster version adds what is needed to distribute it and to recover from failures.

- **Actor-Based Isolation:** the control unit, every keypad, every sensor and every siren is an actor. Each actor processes its messages one at a time and owns its state, so no locks are needed.
- **State Machine with `become()`:** the control unit swaps its message handlers with `getContext().become(...)`. Each state (`recoveryState`, `disarmedState`, `exitDelayState`, `armedState`, `entryDelayState`, `alarmState`) defines exactly which messages it reacts to (Figure 1); a message that a state does not handle is discarded. Two messages are accepted in every state, `RegisterSiren` and `SimulateCrash`: a small helper method, `state(...)`, adds them to the handlers of each state.
- **Non-Blocking Timers:** exit and entry delays use the single-shot timers of `AbstractActorWithTimers`. Both delays use the same timer key, so starting or cancelling one never leaves a stale timeout behind.
- **Cluster Singleton for the Control Unit:** the control unit is started through a `ClusterSingletonManager` on every node with the `control` role. The managers guarantee that exactly one instance runs, on the oldest `control` node. If that node leaves or is removed after a crash, the instance is created again on another `control` node.
- **Location Transparency with a Proxy:** on the other nodes a `ClusterSingletonProxy` is created: a local actor that forwards messages to the control unit wherever it runs. Sensors and keypads only

know this local proxy. While the singleton is being created or moved, the proxy buffers the messages.

- **Serialization:** every message that can cross nodes implements the marker interface `CborSerializable`, which `application.conf` binds to Pekko's Jackson CBOR serializer (Section 3.2). Messages are immutable Java records; sets of zones are copied with `Set.copyOf` when a message is created.
- **Safe Recovery Mode by Construction:** a new instance of an actor always starts with empty fields and its initial behavior. For this reason `createReceive()` returns `recoveryState()`, not `disarmedState()`. Every way of obtaining a new instance (first start, restart by the supervisor after an exception, re-creation on another node) leads to the recovery mode, without any special case in the code.
- **Self-Registering Sirens:** the control unit does not receive the siren in its constructor, because a new instance would not know where the sirens are. Instead, a siren sends `RegisterSiren` to the control unit when it starts and then every 3 seconds, so a new control unit finds it again within a few seconds. Repeated registrations are harmless, since the sirens are kept in a set.
- **Removal of Crashed Nodes:** the Split Brain Resolver removes a node that has been unreachable for a while, so that the singleton can be recreated elsewhere.

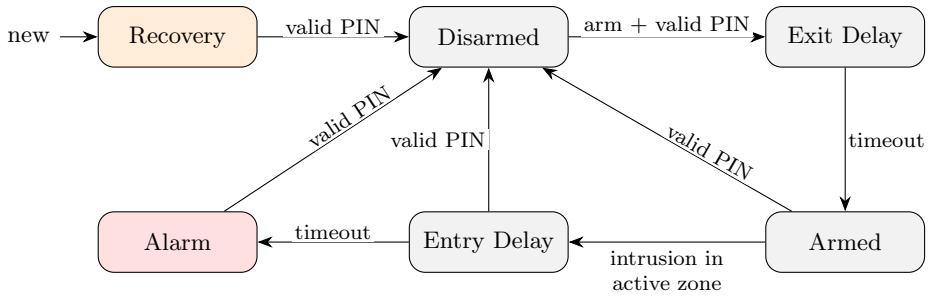


Figure 1: States of the control unit. Every new instance starts in Recovery. Sensor events in Recovery, Disarmed, Exit Delay, Entry Delay and Alarm are only logged; wrong PINs never cause a transition.

3 Implementation Overview

3.1 Components

- **SmartHomeProtocol:** the message protocol. Messages to the control unit are `SensorTriggered(sensorId, zone)`, `ArmSystemRequest(zones)`, `ValidPinEntered`, `InvalidPinEntered`, `RegisterSiren` and `SimulateCrash`. The keypad receives `InsertPin` and `ArmWithPin`, sensors receive `OpenDoorMsg`, `OpenWindowMsg` and `MotionDetectedMsg`, sirens receive `ActivateSiren` and `DeactivateSiren`.
- **ControlUnit:** the state machine of Figure 1. It stores the active zones (an empty set means full arming) and the registered sirens. `SimulateCrash` throws an exception, so that the supervisor restarts the actor: it is used by the tests to show the recovery mode.
- **KeyPad:** validates the PIN. A correct PIN becomes `ValidPinEntered` or, for an arming request, `ArmSystemRequest`; a wrong PIN becomes `InvalidPinEntered`. The system is therefore never armed without the PIN.
- **Sensor:** has a `sensorId` and a zone. It reacts to a door being opened, a window being opened or a motion being detected, each with its own message and handler, by sending `SensorTriggered` to the control unit. Since the event carries the zone, the control unit needs no table mapping sensors to zones, and new sensors can be added on any node without changing it.
- **Siren:** registers itself periodically and switches on and off on request.

3.2 Configuration: `application.conf`

Pekko reads its configuration from the file `application.conf` in `src/main/resources` (HOCON format) when an `ActorSystem` is created. The file contains the settings that turn an ordinary actor system into a cluster node; they are the same for every node.

```
pekko {
  loglevel = "INFO"
  actor {
    provider = "cluster"
    serialization-bindings {
      "org.example.CborSerializable" = jackson-cbor
    }
  }
  serialization.jackson.serialization-features {
    FAIL_ON_EMPTY_BEANS = off
  }
```

```
  remote.artery.canonical.hostname = "127.0.0.1"
  cluster {
    downing-provider-class =
      "org.apache.pekko.cluster.sbr.SplitBrainResolverProvider"
    split-brain-resolver.stable-after = 7s
  }
}
```

- `provider = "cluster"`: by default a Pekko actor system is local: actors live in one process and cannot be reached over the network. With the cluster provider, the actor system becomes a cluster node: it opens a network port, joins other nodes, and an `ActorRef` can point to an actor on another node. Without this line there is no cluster.
- `serialization-bindings`: a message sent to another node must be turned into bytes. Java serialization is disabled by default in Pekko, so a serializer must be chosen explicitly: every message implementing `CborSerializable` is serialized with Jackson in the CBOR (binary JSON) format. The `pekko-serialization-jackson` dependency provides this serializer.
- `FAIL_ON_EMPTY_BEANS = off`: some messages have no fields (e.g. `ActivateSiren`, `OpenDoorMsg`); by default Jackson refuses to serialize them, and this option allows it.
- `remote.artery.canonical.hostname`: the address on which the node listens and through which the others reach it (Artery is Pekko's networking layer). The port is chosen by each node; the tests use a random free port.
- **Split Brain Resolver:** when a node crashes, the others only see that it no longer answers (it becomes *unreachable*); it could be dead, or the network could be temporarily broken, so by default Pekko never removes it. While it is still a member, the singleton is not moved. The Split Brain Resolver decides instead: a node that stays unreachable for `stable-after` (7 s) is removed, and the control unit can be recreated on another `control` node. It also guarantees that, if the network splits, only one side survives, so there is never more than one control unit.

4 Testing

The behaviour of the system, including its distribution over several nodes, is demonstrated by two JUnit 5 test classes, run with `mvn test`. They use the Pekko TestKit (`javadsl.TestKit`) and follow the structure of the tests of the single-node version: a fresh environment for every test, messages sent with `tell`, and assertions on the messages received by the siren.

4.1 Setup

The tests run *real* cluster nodes: several `ActorSystem`s inside the test JVM, each listening on its own port on localhost. Every message between a sensor, the keypad, the control unit and the siren therefore goes through the network and Jackson serialization, exactly as in a deployment on separate machines.

Creating the cluster (`@BeforeEach`). Before every test three nodes are created and joined into a cluster:

1. A common configuration is built on top of `application.conf` (`withFallback`), adding a random free port (`port = 0`), since all the nodes run on the same machine, and an option that allows several nodes in the same JVM.

2. Three `ActorSystem`s are created with the same name, `SmartHome` (only actor systems with the same name can form a cluster), each with its own role: `control`, `sensors` and `keypad`.

3. The control node joins its own address, which means it founds the cluster; the other two nodes join that address.

4. Joining is asynchronous, so the setup waits, with the TestKit's `awaitCond`, until the cluster reports three members in the *Up* state.

Creating the actors (`createEnvironment`). At the beginning of each test the actors are created on their nodes (Figure 2):

1. On the control node, a `ClusterSingletonManager` for the role `control`. It creates the `ControlUnit`, with exit and entry delays of 1 s to keep the tests short. The new control unit starts in Recovery.

2. On the sensors node and on the keypad node, a `ClusterSingletonProxy`, which locates the control unit through the path of the manager (`/user/controlUnit`).

3. On the sensors node, four sensors (front door and kitchen window in the Perimeter zone, living room motion in GroundFloor, bedroom motion in UpperFloor), connected to the local proxy.

4. On the keypad node, the keypad, connected to the local proxy.

5. A TestKit probe on the keypad node plays the role of the siren: it is registered with `RegisterSiren`, so `ActivateSiren` and `DeactivateSiren` reach the probe, where the test can check them.

6. The correct PIN is entered on the keypad, and the setup waits for the probe to receive `DeactivateSiren`. Leaving Recovery is the only transition that sends this message without a previous alarm, so its arrival is the signal that the control unit exists, knows the probe and is now Disarmed. The wait allows up to 20 s, because right after the cluster is formed the singleton may take a moment to start.

The registration is sent before the PIN and through the same proxy, so it reaches the control unit first.

Shutdown (`@AfterEach`). After every test the three nodes are shut down, so each test starts with a new cluster and tests cannot affect each other.

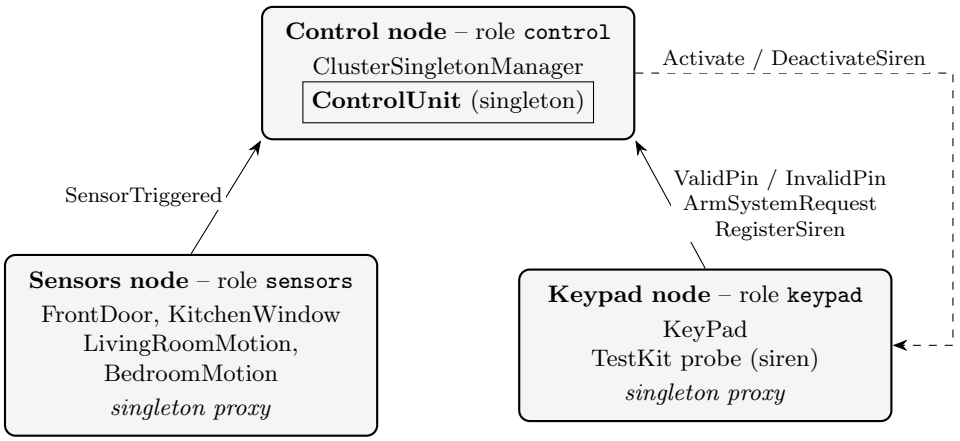


Figure 2: Nodes and actors created by the tests. Sensors and keypad send to the local proxy, which forwards to the ControlUnit; the ControlUnit sends to the siren (the probe) directly.

4.2 How the Tests Check the Behaviour

Each test sends messages as a user or a sensor would (a PIN or an arming request to the keypad, a door, window or motion event to a sensor), waits with `Thread.sleep` when a delay has to expire, and then checks what the siren probe receives: `expectMsgClass` when the siren must be activated or deactivated within a given time, `expectNoMessage` when nothing must happen during a given interval. The probe is the only observation point, so a state is verified through its effect on the siren: for example, the system is Armed if a sensor in an active zone leads to `ActivateSiren` after the entry delay.

4.3 Test Scenarios

`SmartHomeClusterTest` (three nodes: `control`, `sensors`, `keypad`):

- **testSirenFiresOnTimeout:** an intrusion in an active zone fires the siren after the entry delay, and not before.
- **testSuccessfulDisarmDuringEntryDelay:** the correct PIN during the entry delay prevents the alarm.
- **testPartialArmingIgnoresInactiveZones:** in night mode the bedroom sensor is ignored, while a window on the perimeter fires the siren.
- **testSensorsIgnoredWhenDisarmed** and **testSensorsIgnoredDuringExitDelay:** sensor events do not fire the siren.
- **testAlarmStopsWithValidPin** and **testInvalidPinDuringAlarm:** during the alarm only the correct PIN deactivates the siren.
- **testFullArmingActivatesAllZones:** arming with no zones activates all of them, including the upper floor.
- **testArmingWithWrongPinIsRejected:** a wrong PIN does not arm the system.
- **testDisarmFromArmedState:** the correct PIN disarms the system while it is Armed.
- **testRecoveryAfterCrash:** while Armed, the control unit crashes and is restarted by its supervisor. The new instance ignores an intrusion (it does not assume Armed), refuses to arm (it does not assume Disarmed) and ignores a wrong PIN; the correct PIN switches the siren off and leads to Disarmed, after which the system works normally.

`ControlUnitFailoverTest` (nodes A: `control`; B: `control`, `sensors`; C: `keypad`). The setup is the same, with two `control` nodes, each with a singleton manager; node A founds the cluster, so it is the oldest and hosts the control unit.

- **testControlUnitRecreatedOnAnotherNode:** with the system Armed, node A leaves the cluster. The control unit is recreated on node B; it ignores an intrusion, and the correct PIN produces `DeactivateSiren`, which shows that the new instance was in Recovery.

5 Conclusion

The actor model handles the concurrency of the alarm system without locks: each component owns its state, the control unit is a state machine driven by

messages, and timeouts are messages too. The cluster version keeps the same alarm logic and adds three ingredients: a Cluster Singleton that guarantees a single control unit and recreates it after failures, a proxy on every node that makes its location transparent, and a recovery mode that every new instance enters by construction, so that the loss of the previous state never leads the system to assume it is armed or disarmed.

Known limitations. The tests use fixed waits (`Thread.sleep`) and check the state only through the siren, like the tests of the single-node version: on a very slow machine the waits may need to be longer. Since every test forms a new cluster, the whole suite takes about one or two minutes. After a control unit is recreated, a siren may need up to 3 seconds to register again; an alarm is not possible in that window anyway, since the new instance is in Recovery until the PIN is entered. If a node crashes rather than leaving gracefully, the singleton is recreated only after the Split Brain Resolver has removed the node (7 seconds of unreachability in the provided configuration).